

Knowledge representation

Chapter 5

Knowledge Representation

- knowledge representation (KR) in AI refers to **encoding information about the world into formats that AI systems can utilize to solve complex tasks.**
- This process enables machines to reason, learn, and make decisions by structuring data in a way that mirrors human understanding.
- The objective of knowledge representation is to express the knowledge about the world in a computer-tractable form.
- Key aspects
 - **Syntax** : Defines how sentences are formed in a language
 - **Semantics**: describes the meaning of sentences, what is it the sentence refers to in the real world.
 - **Computational aspect** : describes how sentences and objects are manipulated in concordance with semantical conventions.

Knowledge representation using Logic

- Logic is defined as a formal language for expressing knowledge and ways of reasoning.
- For example, in the language of arithmetic,
- $X + 2 > y$ is true iff the number $x + 2$ is no less than the number y .
- $X + 2 > y$ is true in a world where $x = 7$; $y = 1$.
- $X + 2 > y$ is false in a world where $x = 0$; $y = 6$.
- Knowledge representation using logic shows the following:
 - i. How logic can be used to form representation of the world ?
 - ii. How a process of inference can be used to derive new representations about the world ?
 - iii. How these can be used by an intelligent agent to deduce what to do ?

Types of Logic

- Different types of logic are :
 - Propositional Logic
 - First-Order Predicate Logic

Propositional Logic

- A **propositional logic** is a declarative sentence which can be either true or false but not both.
- These propositions represent facts or conditions about a situation.
- We use symbols to represent the propositions and logical operations to connect those propositions.
- Examples of propositional logic:
 - P : “ The sky is blue. “ (This can be either true or false).
 - Q: “ It is raining right now.” (This can also be true or false).

Basic concepts of propositional Logic

- Propositions
 - A proposition is a statement that can either be true or false. It does not matter how complicated statement is if it can be classified as true or false then it is a proposition.
 - For example:
 - The sky is blue. (True)
 - It is raining. (False)
- Logical Connectives
 - AND ($\wedge$): This operation is true if both propositions are true.
 - Example: "It is sunny $\wedge$ it is warm" is true only if both "It is sunny" and "It is warm" are true.
 - OR ($\vee$): This operation is true if at least one of the propositions is true.
 - Example: "It is sunny $\vee$ it is raining" is true if either "It is sunny" or "It is raining" is true.
 - NOT ($\neg$): This operation reverses the truth value of a proposition.
 - Example: " $\neg$ It is raining" is true if "It is raining" is false.
 - IMPLIES ($\rightarrow$): This operation is true if the first proposition leads to the second.
 - Example: "If it rains then the ground is wet" (It rains $\rightarrow$ The ground is wet) is true unless it rains and the ground is not wet.
 - IF AND ONLY IF ($\leftrightarrow$): This operation is true if both propositions are either true or false together.
 - Example: "It is raining $\leftrightarrow$ The ground is wet" is true if both "It is raining" and "The ground is wet" are either true or both false.

Basic concepts of propositional Logic

- Truth tables
 - They are used to find the truth value of complex propositions by checking all possible combinations of truth values for their components.
 - They systematically list every possible combinations which helps in making it easy to find how different logical operators affect the overall outcome.
- Tautologies, Contradictions and Contingencies
 - **Tautology:** A proposition that is always true no matter the truth values of the individual components.
Example: " $P \vee \neg P$ " (This is always true because either P is true or P is false).
 - **Contradiction:** A proposition that is always false.
Example: " $P \wedge \neg P$ " (This is always false because P can't be both true and false at the same time).
 - **Contingency :** A proposition that can be true or false depending on the truth values of its components.
 - Example: " $P \wedge Q$ " (This is true only if both P and Q are true).

Properties of operators

- **Commutativity** : Order of propositions doesn't matter when using AND ($\wedge$) or OR ($\vee$).
 - $P \wedge Q \equiv Q \wedge P$
 - $P \vee Q \equiv Q \vee P$
- **Associativity**: Grouping of propositions doesn't matter when using multiple ANDs or ORs.
 - $(P \wedge Q) \wedge R \equiv P \wedge (Q \wedge R)$
 - $(P \vee Q) \vee R \equiv P \vee (Q \vee R)$
- **Distributivity**: AND ($\wedge$) and OR ($\vee$) can distribute over each other which is similar to multiplication and addition in math.
 - $P \wedge (Q \vee R) \equiv (P \wedge Q) \vee (P \wedge R)$
 - $P \vee (Q \wedge R) \equiv (P \vee Q) \wedge (P \vee R)$
- **Identity**: A proposition combined with "True" or "False" behaves predictably.
 - $P \wedge \text{true} \equiv P$
 - $P \vee \text{false} \equiv P$
- **Domination**: When combined with "True" or "False" some outcomes are always fixed.
 - $P \vee \text{true} \equiv \text{true}$
 - $P \wedge \text{false} \equiv \text{false}$
- **Double Negation**: Negating a proposition twice cancels out the negation.
 - $\neg(\neg P) \equiv P$
- **Idempotence**: Repeating same proposition with AND or OR doesn't change its value.
 - $P \wedge P \equiv P$
 - $P \vee P \equiv P$

Inference rules in propositional Logic

Inference rules in propositional logic:

Rule of inference	Tautology	Name
$\begin{array}{l} p \rightarrow q \\ p \\ \hline \therefore q \end{array}$	$[p \wedge (p \rightarrow q)] \rightarrow q$	Modus ponens
$\begin{array}{l} \neg q \\ p \rightarrow q \\ \hline \therefore \neg p \end{array}$	$[\neg q \wedge (p \rightarrow q)] \rightarrow \neg p$	Modus tollens
$\begin{array}{l} p \rightarrow q \\ q \rightarrow r \\ \hline \therefore p \rightarrow r \end{array}$	$[(p \rightarrow q) \wedge (q \rightarrow r)] \rightarrow (p \rightarrow r)$	Hypothetical syllogism
$\begin{array}{l} p \vee q \\ \neg p \\ \hline \therefore q \end{array}$	$((p \vee q) \wedge \neg p) \rightarrow q$	Disjunctive syllogism
$\begin{array}{l} p \\ \hline \therefore p \vee q \end{array}$	$p \rightarrow (p \vee q)$	Addition
$\begin{array}{l} p \wedge q \\ \hline \therefore p \end{array}$	$(p \wedge q) \rightarrow p$	Simplification
$\begin{array}{l} p \\ q \\ \hline \therefore p \wedge q \end{array}$	$((p) \wedge (q)) \rightarrow (p \wedge q)$	Conjunction
$\begin{array}{l} p \vee q \\ \neg p \vee r \\ \hline \therefore q \vee r \end{array}$	$[(p \vee q) \wedge (\neg p \vee r)] \rightarrow (p \vee r)$	Resolution

Proof of Tautology

- $[p \wedge (p \rightarrow q)] \rightarrow q$
- Let's create a truth table for this statement

-

P	Q	$P \rightarrow Q$	$P \wedge (P \rightarrow Q)$	$[P \wedge (P \rightarrow Q)] \rightarrow Q$
0	0	1	0	1
0	1	1	0	1
1	0	0	0	1
1	1	1	1	1

- In this way, we can say that the law of Modus ponens is True.

Proof of Tautology

- Now, prove the law of disjunctive syllogism to be true.

Examples

1. Modus Ponens

If it rains, the ground gets wet.
It is raining.
∴ The ground is wet.

2. Modus Tollens

If it rains, the ground gets wet.
The ground is not wet.
∴ It is not raining.

3. Hypothetical Syllogism

If I study, I pass the exam.
If I pass the exam, I get a job.
∴ If I study, I get a job.

4. Disjunctive Syllogism

I will have tea or coffee.
I will not have tea.
∴ I will have coffee.

5. Addition

I passed the exam.
∴ I passed the exam **or** I won a prize.

6. Simplification

I am studying and listening to music.
∴ I am studying.

7. Conjunction

It is cold.
It is raining.
∴ It is cold and raining.

8. Resolution

I will go to the movie or stay home.
I will not go to the movie or I will study.
∴ I will stay home or I will study.

Predicate Logic

- Also known as First Order Logic.
- It helps us represent knowledge, reason through problems and understand languages.
- By building on propositional logic and adding quantifiers and predicates, FOL allows us to express more complex relationship and make decisions based on logical reasoning.

Key components of First-Order Logic

- FOL extends propositional logic by introducing quantifiers and predicates, making them more expressive and versatile.
- Key components:
- Constants: These represent specific objects or entities. (Example: Ashok, 13, NCMT)
- Variables: These stand for unspecified objects or entities. (Example: x , y , z)
- Predicates: These define properties or relationships. (Example: Likes(Alice, Bob) means “Alice likes Bob”).

Key components of First-Order Logic

- Functions: It map objects to other objects(Example: MotherOf(x) refers to the mother of x).
- Quantifiers: These define the scope of the variables.
 - **Universal quantifiers($\forall$)**: Applies a predicate to all elements (Example: $\forall x$ (Person(x) $\rightarrow$ Mortal(x)) means "All persons are mortal").
 - **Existential quantifiers($\exists$)**: Shows the existence of at least one element. (Example: $\exists x$ (Person(x) $\wedge$ Likes(x, IceCream)) means "Someone likes ice cream").
- Logical Connectives: Include conjunction ($\wedge$), disjunction ($\vee$), implication ($\rightarrow$), biconditional ($\leftrightarrow$) and negation ($\neg$).

Conversion to FOPL

- All humans are mortals.
 - $\forall x(\text{Human}(x) \rightarrow \text{Mortal}(x))$
 - How to interpret?
 - For every x, if x is a human, then x is mortal.
- Every bird has wings.
 - $\forall x(\text{Bird}(x) \rightarrow \text{HasWings}(x))$
 - How to interpret ?
 - For every x, if x is a bird, then it has wings.
- Some student is intelligent
 - $\exists x(\text{Student}(x) \wedge \text{Intelligent}(x))$
 - There exists at least one x such that x is a student and intelligent.
- There is a black dog.
 - **TRY IT YOURSELF !!!**

Conversion to FOPL

- Example 1:
 - All teachers who are experienced are respected.
 - $\forall x((\text{Teacher}(x) \wedge \text{Experienced}(x)) \rightarrow \text{Respected}(x))$
- Example 2:
 - Ram is a student of AI class.
- Example 3
 - Some students like Mathematics
 - Every student is not lazy
 - Every patient who has fever and cough has flu.
 - Every student likes some book.

Order of Quantifiers

- Brothers are siblings.
 - $\forall x \forall y \text{ Brother}(x,y) \rightarrow \text{Sibling}(x,y)$
- Every student has a book
 - Scenario 1:
 - Each student may have a different book.
 - $\forall x \exists y (\text{Student}(x) \rightarrow \text{Has}(x,y))$
 - For every student x , there exists **some book y** that x has.
 - Scenario 2:
 - There is one single book for all students.
 - $\exists y \forall x (\text{Student}(x) \rightarrow \text{Has}(x,y))$
 - There exists one book that every student has.
 - Order of quantifiers matters.
- Every Student likes a teacher.
 - Scenario 1
 - Scenario 2

Rules of Inference

- Propositional Logic
 - Deal with statements that are true or false
 - **Modus Ponens**
 - If a conditional statement ("if-then" statement) is true, and its antecedent (the "if" part) is true, then its consequent (the "then" part) must also be true.
 - If $p \rightarrow q$ and p , then q .
- *Given $(P \vee Q) \wedge (\neg P \vee R) \therefore Q \vee R$*
 - *Step 1: $(P \vee Q)$ (From simplification)*
 - *Step 2: $(\neg P \vee R)$ (From simplification)*
 - *Step 3: $Q \vee R$ (From 1 and 2, Resolution)*

Rules of Inference

Rules of Inference

Rules of Inference	Form	Meaning	Example
Universal instantiation	$\forall x P(x) \Rightarrow P(c)$	If something is true for all x , it's true for a particular case c .	All humans are Mortal. If its true for everyone, then true for X . Then, X is mortal.
Universal generalization	$P(c) \Rightarrow \forall x P(x)$	If something is true for any arbitrary element, it's true for all.	Suppose you pick a random number x , and show: $X + 0 = X$ Then, we can conclude: For all X , $X + 0 = X$
Existential instantiation	$\exists x P(x) \Rightarrow P(c)$	If something exists, we can give it a name (c).	There exists a student who passed the exam. Let Alice be the student who passed the exam
Existential generalization	$P(c) \Rightarrow \exists x P(x)$	If something is true for a particular c , it's true for "some x ".	Socrates is a philosopher. Then, we can say, There exists a philosopher.

Rules of Inference

- First Order Predicate Logic

- Example 1

- All humans are mortal.
 - All Greeks are humans.
 - Socrates is a Greek.
 - Therefore, Socrates is mortal.

- Solution:

- $\forall x (\text{Human}(x) \rightarrow \text{Mortal}(x))$ -----(1)
 - $\forall x (\text{Greek}(x) \rightarrow \text{Human}(x))$ -----(2)
 - $\text{Greek}(\text{Socrates})$ -----(3)
 - Goal: **Mortal(Socrates)**

Step 1: Universal Instantiation on (1)

$\forall x (\text{Greek}(x) \rightarrow \text{Human}(x))$

Instantiate with Socrates:

Greek(Socrates) $\rightarrow$ Human(Socrates) ----- (4)

Step 2: Modus Ponens using 3 and 4

Human(Socrates) ----- (5)

Step 3: Universal Instantiation on (2)

Human(Socrates) $\rightarrow$ Mortal(Socrates) ----- (6)

Step 4: Modus Ponens using (5) and (6)

Mortal(Socrates) ----- (7)

Hence, final conclusion is proved.

Forward Chaining

- Forward chaining is a data-driven inference technique.
- It starts with the available data and applies rules to infer new data until a goal is reached.
- How does it work ??
 1. Start with known facts : The inference engine begins with the known facts in the knowledge base.
 2. Apply Rules: It looks for rules whose conditions are satisfied by the known facts.
 3. Infer new facts: When a rule is applied, new facts are inferred and added to the knowledge base.
 4. Repeat: This process is repeated until no more rules can be applied or a specified goal is achieved.

Forward Chaining

- Rules:
 - $\forall x (\text{Student}(x) \wedge \text{Studies}(x) \rightarrow \text{Passes}(x))$
 - $\forall x (\text{Passes}(x) \rightarrow \text{Graduates}(x))$
- Facts
 - $\text{Student}(\text{Alice})$
 - $\text{Studies}(\text{Alice})$
- Forward Chaining Steps
 - Step 1: Initial Facts
 - $\text{Student}(\text{Alice})$
 - $\text{Studies}(\text{Alice})$
 - Step 2: Use Rule (1)
 - $\text{Student}(\text{Alice}) \wedge \text{Studies}(\text{Alice})$ is True
 - New Fact: $\text{Passes}(\text{Alice})$
 - Step 3: Update Facts
 - $\text{Student}(\text{Alice})$
 - $\text{Studies}(\text{Alice})$
 - $\text{Passes}(\text{Alice})$
 - Step 4: Use Rule (2)
 - $\text{Passes}(\text{Alice})$ is True.
 - New Fact: $\text{Graduate}(\text{Alice})$
 - Step 5: Update new fact:
 - $\text{Graduates}(\text{Alice})$

Forward Chaining Example

- **Rules**

- *If a person has both fever and cough, they are suspected of having the flu.
Anyone suspected of flu should rest.
Anyone suspected of flu should be recommended fluids.
If a person is high-risk and is suspected of flu, they should visit a doctor.*

- **Facts**

- John has fever.
- John has cough.
- John is at high risk.

Forward Chaining Example

- *If a person has both fever and cough, they are suspected of having the flu.
Anyone suspected of flu should rest.
Anyone suspected of flu should be recommended fluids.
If a person is high-risk and is suspected of flu, they should visit a doctor.*
- John has fever.
- John has cough.
- John is at high risk.
- Rules
 - $\forall x (\text{Fever}(x) \wedge \text{Cough}(x) \rightarrow \text{SuspectFlu}(x))$
 - $\forall x (\text{SuspectFlu}(x) \rightarrow \text{RecommendRest}(x))$
 - $\forall x (\text{SuspectFlu}(x) \rightarrow \text{RecommendFluids}(x))$
 - $\forall x (\text{HighRisk}(x) \wedge \text{SuspectFlu}(x) \rightarrow \text{RecommendDoctorVisit}(x))$
- Facts
 - $\text{Fever}(\text{John})$
 - $\text{Cough}(\text{John})$
 - $\text{HighRisk}(\text{John})$
- Try to infer facts from the above rules and given facts.

Forward Chaining Example

- Initial Fact base
 - Fever(John)
 - Cough(John)
 - HighRisk(John)
- Match rules against facts
 - Use rule no. 1
 - Fever(John)
 - Cough(John)
 - New Fact: SuspectFlu(John)
- Update Fact base: SuspectFlu(John)
- Rule R2 triggers: RecommendRest(John)
- Rule R3 Triggers: RecommendFluids(John)
- Check For rule R4:
 - HighRisk(John)
 - SuspectFlu(John)
 - RecommendDoctorVisit(John)

Forward Chaining

- **Facts:**

- West is an American.
- Nono is an enemy of America.
- M1 is a missile.
- Nono owns M1.

- **Rules**

- If a person is American and sells a weapon to an enemy of America, then that person is a criminal.
- Every missile is a weapon.
- If Nono owns a missile, then West sold that missile to Nono.

- **Goal**

- **West is criminal.**

Forward Chaining

- American(West)
- Enemy(Nono, America)
- Missile(M1)
- Owns(Nono, M1)
- $\forall x \forall y \forall z [\text{American}(x) \wedge \text{Weapon}(y) \wedge \text{Sells}(x,y,z) \wedge \text{Enemy}(z,\text{America}) \rightarrow \text{Criminal}(x)]$
- $\forall x [\text{Missile}(x) \rightarrow \text{Weapon}(x)]$
- $\forall x [\text{Missile}(x) \wedge \text{Owns}(\text{Nono},x) \rightarrow \text{Sells}(\text{West},x,\text{Nono})]$

Forward Chaining

- We know, Missile(M1)
- Using Rule 2, Weapon(M1)
- We know, Missile(M1) and Owns(Nono, M1),
- Using R3, Sells(West, M1, Nono)
- We have, American(West)
- Weapon(M1)
- Sells(West,M1,Nono)
- Enemy(Nono,America)
- Apply R1, Criminal(West)
- Keep firing the rules until you get to the goal.

Forward Chaining

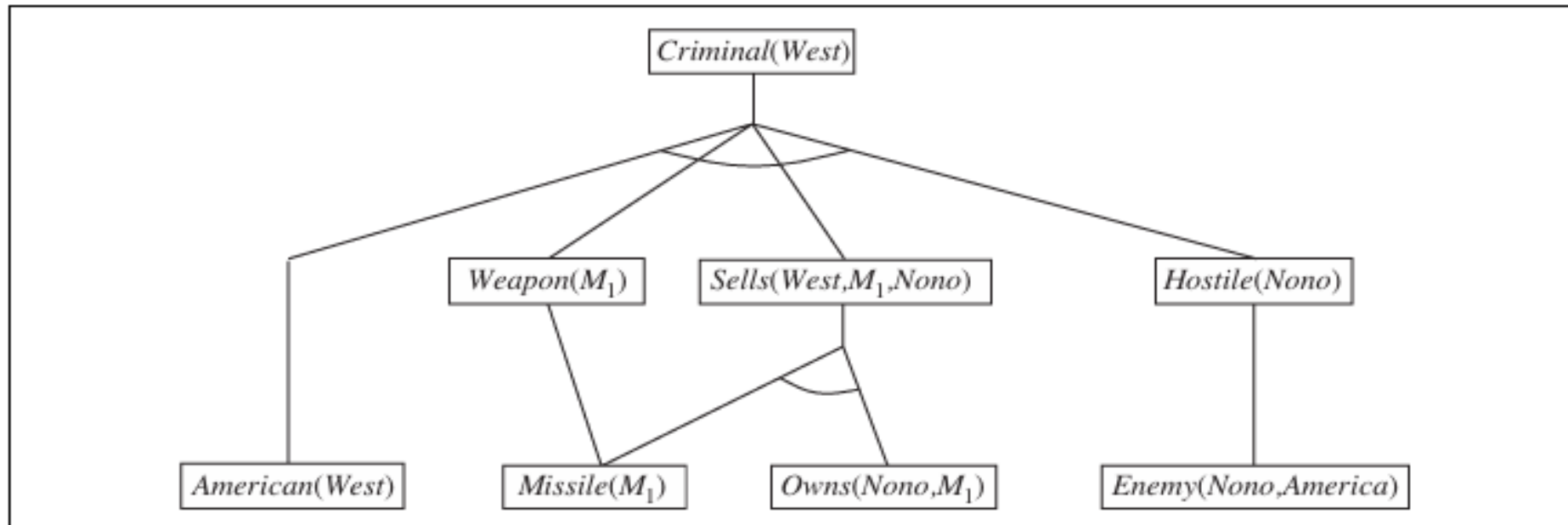


Figure 9.4 The proof tree generated by forward chaining on the crime example. The initial facts appear at the bottom level, facts inferred on the first iteration in the middle level, and facts inferred on the second iteration at the top level.

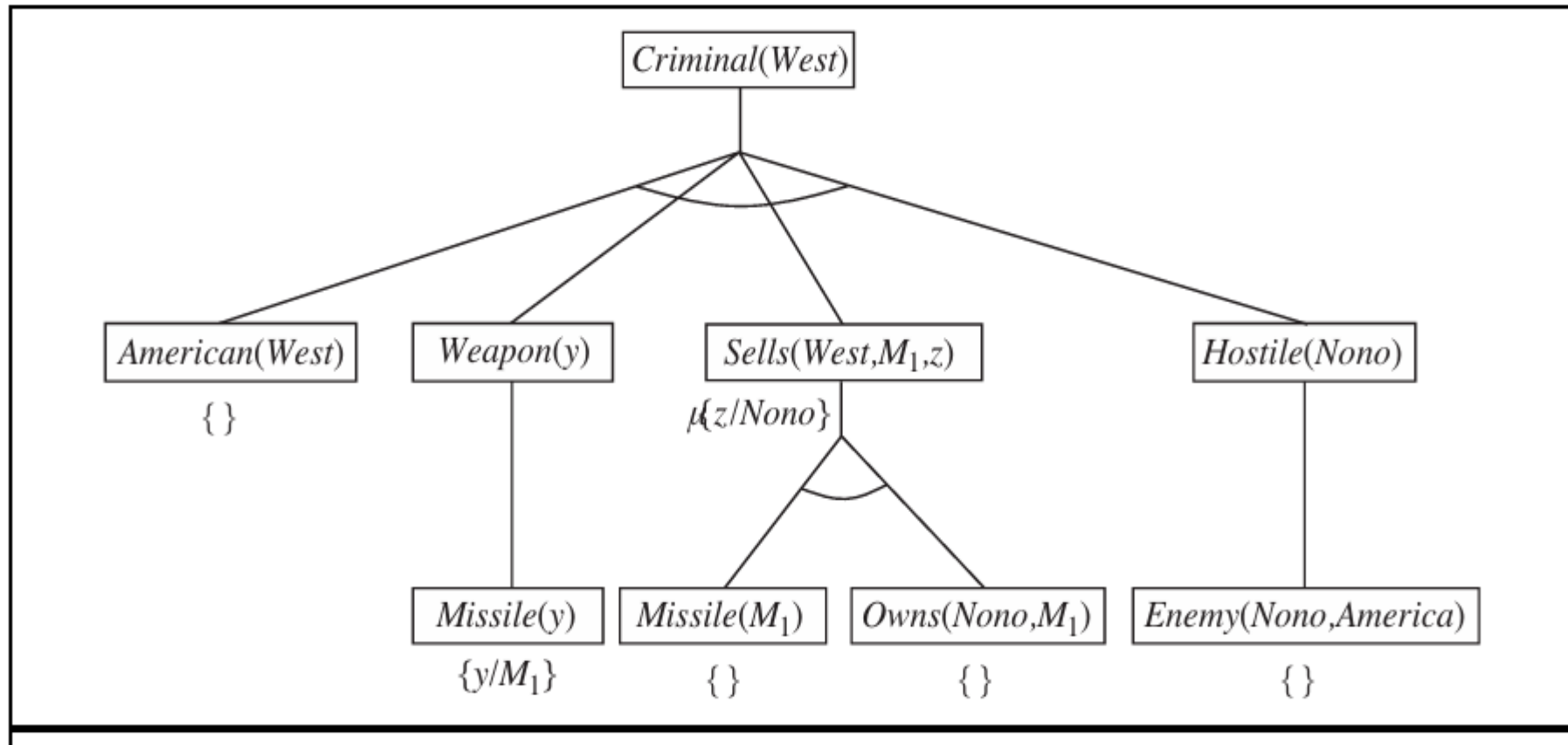
Backward Chaining

- Backward chaining is a goal-driven inference technique.
- It starts with the goal and works backward to determine which facts must be true to achieve that goal.
- This method is ideal for situations where the goal is clearly defined, and the path to reach it needs to be established.
- Steps Involved:
 - **Start with a Goal:** The inference engine begins with the goal or hypothesis it wants to prove.
 - **Identify Rules:** It looks for rules that could conclude the goal.
 - **Check Conditions:** For each rule, it checks if the conditions are met, which may involve proving additional sub-goals.
 - **Recursive Process:** This process is recursive, working backward through the rule set until the initial facts are reached or the goal is deemed unattainable.

Backward Chaining

- Start with the goal
 - Criminal(West)
- What rule includes the criminal ?
 - Rule R1:
 - What does it need ? ?
 - American(West)
 - Weapon(y)
 - Sells(West,y,z)
 - Enemy(z,America)
 - Check subgoals?
 - American(West) (Done)
 - Enemy(None, America) (Done)
 - Need to prove Weapon(M1) and Sells(West, M1, Nono) (y = M1, and z = Nono)
 - Step 3
 - To prove **Weapon(M1)**:
 - **Rule R2 we need, Missile(M1) (Already True)**
 - Step 4:
 - **Rule R3, we need: Missile(M1) and Owns(Nono, M1) (Both satisfied)**
 - **Thus, Criminal(west) is proved.**

Backward Chaining



Unification

- Unification is a key concept in First-order logic(FOL) that involves finding a common substitution for two logical expressions, making them identical.
- This process is crucial for automated reasoning, theorem proving and various inference methods in artificial intelligence.
- Example: Let us say we have the following predicates – $P(x, \text{Dog})$ and $P(\text{Alex}, y)$. To unify them, we must find substitutions such that both predicates become the same. The solution is –
 - $X \rightarrow \text{Alex}$
 - $Y \rightarrow \text{Dog}$
- After substitution, both predicates become $P(\text{Alex}, \text{Dog})$, meaning they are unified.

Implementation of Unification algorithm

- Determine the Most General Unifier(MGU) between WorksAt(Google, X, Manager) and WorksAt(Google, John, Y).
- **Step 1:** Start with an empty set of substitutes.
- **Step 2:** Combine the atomic statements step by step.
- The predicate "WorksAt" is identical, so we can proceed.
- The first argument "Google" matches in both cases.
- For the second argument, "X" is a variable, so we substitute X with John.
- For the third parameter, "Manager" and "Y": since Y is a variable, we substitute Y with Manager.
- Unified Expression: WorksAt (Google, John, Manager). Unifier: {X/John, Y/Manager}.

Converting to CNF

- **Conjunctive Normal Form (CNF)** is a logical sentence written as:
 - A conjunction (AND) of clauses
where each clause is a disjunction (OR) of literals.
- Step 1: Eliminate biconditionals and implications
 - Eliminate $\Leftrightarrow$, replacing $\alpha \Leftrightarrow \beta$ with $(\alpha \Rightarrow \beta) \wedge (\beta \Rightarrow \alpha)$.
 - Eliminate $\Rightarrow$, replacing $\alpha \Rightarrow \beta$ with $\neg\alpha \vee \beta$.
- Step 2: Move $\neg$ inwards:
 - $\neg(\forall x p) \equiv \exists x \neg p$
 - $\neg(\exists x p) \equiv \forall x \neg p$
 - $\neg(\alpha \vee \beta) \equiv \neg\alpha \wedge \neg\beta$
 - $\neg(\alpha \wedge \beta) \equiv \neg\alpha \vee \neg\beta$
 - $\neg\neg\alpha \equiv \alpha$

Converting to CNF

- Step 3: Standardize variables apart by renaming them: each quantifier should use a different variable.
 - $\forall x \text{ Loves}(x, \text{IceCream})$
 - $\forall y \text{ Hates}(y, \text{Spinach})$
- Step 4: Distribute $\wedge$ over $\vee$:
 - $(\alpha \wedge \beta) \vee \gamma \equiv (\alpha \vee \gamma) \wedge (\beta \vee \gamma).$

Converting to CNF

- **Step 5: Skolemization** :- Each existential variables is replaced by Skolem constant or Skolem function of enclosing universally quantified variables.
- Case I : Existential Quantifier alone (use Skolem constant)
 - For instance, $\exists x \text{ Rich}(x)$ becomes $\text{Rich}(G1)$ where $G1$ is a new Skolem constant.
 - $\exists y (\text{Cat}(y) \wedge \text{Black}(y))$
 - After Skolemization: $\text{Cat}(b) \wedge \text{Black}(b)$ where b is a skolem constant.
- Case II: Universal Quantifier Alone
 - $\forall x \text{ Human}(x) \rightarrow \text{Mortal}(x)$
 - $\forall x (\neg \text{Human}(x) \vee \text{Mortal}(x))$
 - $\neg \text{Human}(x) \vee \text{Mortal}(x)$
- Case III: Existential Inside Universal (use Skolem Function)
 - Example 1: $\forall x \exists y \text{ Parent}(x, y)$
 - Skolemization:
 - $\forall x \text{ Parent}(x, f(x))$
 - $\text{Parent}(x, f(x))$
 - Where, $f(x)$ is a skolem function and y depends on the value of x .
 - Example 2: $\forall x \exists y (\text{Loves}(x, y) \wedge \text{Kind}(y))$
 - Skolemization: $\forall x (\text{Loves}(x, f(x)) \wedge \text{Kind}(f(x)))$
 - $\text{Loves}(x, f(x)) \wedge \text{Kind}(f(x))$

Resolution

- Resolution algorithm is a rule used in AI for logical reasoning.
- It helps our AI system to figure out if the given statement is logically proven from a set of known facts or not.
- It operates mainly on the statements expressed in Conjunctive Normal Form(CNF) and commonly used in Propositional Logic or First Order Predicate Logic.

Working of Resolution Algorithm

- **Convert statements into Logical Forms**

- First, input sentence is converted into a standard format called Conjunctive Normal Form (CNF) means we break down the complex statements into simple parts connected by "AND" and "OR."
- If the original statement "If it is raining then the ground is wet", then its CNF is: "Not(Raining) or Wet".

- **Combine clauses using resolution rule**

- The core idea of the Resolution Algorithm is the resolution rule which combines two clauses to produce a new clause.
- If you have two clauses say A or B and NOT(A) or C you can combine them to get B or C. This process eliminates one variable (A in this case) and simplifies the problem.

- **Repeat until you find an answer**

- The algorithm keeps applying the resolution rule to pairs of clauses until one of two things happens:
- **Contradiction Found** : If the algorithm produces an empty clause (written as FALSE) it means the original set of statements is inconsistent and cannot be true at the same time.
- **No Contradiction** : If no empty clause is found after trying all combinations then the statements are consistent and can coexist..

Example 1:

- Every child loves Santa.
- Everyone who loves Santa loves any Reindeer.
- Rudolph is a reindeer and has a red nose.
- Anything with a red nose is weird or a clown.
- No reindeer is a clown.
- Scrooge does not love anything weird.
- Conclusion: Scrooge is not a child.

Example 1 contd.

- $\forall x \text{Child}(x) \rightarrow \text{Loves}(x, \text{Santa})$
- $\forall x \forall y (\text{Loves}(x, \text{Santa}) \wedge \text{Reindeer}(y) \rightarrow \text{Loves}(x, y))$
- $\text{Reindeer}(\text{Rudolph})$
- $\text{RedNose}(\text{Rudolph})$
- $\forall x \text{RedNose}(x) \rightarrow (\text{Weird}(x) \vee \text{Clown}(x))$
- $\forall x \text{Reindeer}(x) \rightarrow \neg \text{Clown}(x)$
- $\forall x \text{Weird}(x) \rightarrow \neg \text{Loves}(\text{Scrooge}, x)$
- Conclusion: $\neg \text{Child}(\text{Scrooge})$

Conversion to CNF

- $\forall x \text{Child}(x) \rightarrow \text{Loves}(x, \text{Santa})$
 - $\neg \text{Child}(x) \vee \text{Loves}(x, \text{Santa})$ ----- (1)
- $\forall x \forall y (\text{Loves}(x, \text{Santa}) \wedge \text{Reindeer}(y) \rightarrow \text{Loves}(x, y))$
 - $\neg \text{Loves}(x, \text{Santa}) \vee \neg \text{Reindeer}(y) \vee \text{Loves}(x, y)$ ----- (2)
- $\text{Reindeer}(\text{Rudolph})$ ----- (3)
- $\text{RedNose}(\text{Rudolph})$ ----- (4)
- $\forall x \text{RedNose}(x) \rightarrow (\text{Weird}(x) \vee \text{Clown}(x))$
 - $\neg \text{RedNose}(x) \vee \text{Weird}(x) \vee \text{Clown}(x)$ ----- (5)
- $\forall x \text{Reindeer}(x) \rightarrow \neg \text{Clown}(x)$
 - $\neg \text{Reindeer}(x) \vee \neg \text{Clown}(x)$ ----- (6)
- $\forall x \text{Weird}(x) \rightarrow \neg \text{Loves}(\text{Scrooge}, x)$
 - $\neg \text{Weird}(x) \vee \neg \text{Loves}(\text{Scrooge}, x)$ ----- (7)
- $\neg \text{Child}(\text{Scrooge})$
 - Negate it, $\text{Child}(\text{Scrooge})$ ----- (8)

Resolution Proof

- Step 1:
 - $\text{Child}(\text{Scrooge})$ (from 8)
 - Resolve with Clause 1: $\neg\text{Child}(x) \vee \text{Loves}(x, \text{Santa})$
 - Unify $x = \text{Scrooge}$
 - We get, $\text{Loves}(\text{Scrooge}, \text{Santa})$ ----- (9)
- Step 2:
 - $\neg\text{Loves}(x, \text{Santa}) \vee \neg\text{Reindeer}(y) \vee \text{Loves}(x, y)$ (from 2)
 - Unify $x = \text{Scrooge}$, $y = \text{Rudolph}$
 - Resolve with: $\text{Loves}(\text{Scrooge}, \text{Santa})$, $\text{Reindeer}(\text{Rudolph})$
 - $\text{Loves}(\text{Scrooge}, \text{Rudolph})$ ----- (10)
- Step 3:
 - From clause 4, $\text{RedNose}(\text{Rudolph})$
 - Resolve with Clause 5: $\neg\text{RedNose}(x) \vee \text{Weird}(x) \vee \text{Clown}(x)$
 - Unify $x = \text{Rudolph}$
 - $\text{Weird}(\text{Rudolph}) \vee \text{Clown}(\text{Rudolph})$ ----- (11)
- Step 4:
 - $\text{Reindeer}(\text{Rudolph})$ (from clause 3)
 - Resolve with Clause 6, $\neg\text{Reindeer}(x) \vee \neg\text{Clown}(x)$
 - Unify $x = \text{Rudolph}$
 - $\neg\text{Clown}(\text{Rudolph})$ ----- (12)

Resolution Proof

- Step 5:
 - $\text{Weird}(\text{Rudolph}) \vee \text{Clown}(\text{Rudolph})$ (from 11)
 - Resolve with $\neg \text{Clown}(\text{Rudolph})$
 - We get, $\text{Weird}(\text{Rudolph})$ ----- (13)
- Step 6:
 - $\text{Weird}(\text{Rudolph})$ (from 13)
 - Resolve with clause 7: $\neg \text{Weird}(x) \vee \neg \text{Loves}(\text{Scrooge}, x)$
 - Unify $x = \text{Rudolph}$
 - We get, $\neg \text{Loves}(\text{Scrooge}, \text{Rudolph})$ ----- (14)
- Step 7:
 - $\neg \text{Loves}(\text{Scrooge}, \text{Rudolph})$ (from 14)
 - Resolve with $\text{Loves}(\text{Scrooge}, \text{Rudolph})$ (from 10)
 - We derive , $\square$ (empty clause)
- Conclusion:
 - We derived the contradiction.
 - Therefore, the negated contradiction is false.
 - We can say that, Scrooge is not a child.

Example 2:Classwork

- Consider the given facts:
 - All dogs are animals.
 - Fido is a dog.
 - All animals will die.
 - Prove “Fido will die” using resolution.

Example 2:

Consider the following facts:

1. John likes all kinds of food.
 2. Apples are food.
 3. Anything anyone eats and is not killed by is food.
 4. Bill eats peanuts and is still alive.
 5. Sue eats everything Bill eats.
- a) Convert these facts to wffs in predicate logic.
- b) Using resolution, prove that “John likes peanuts”.